

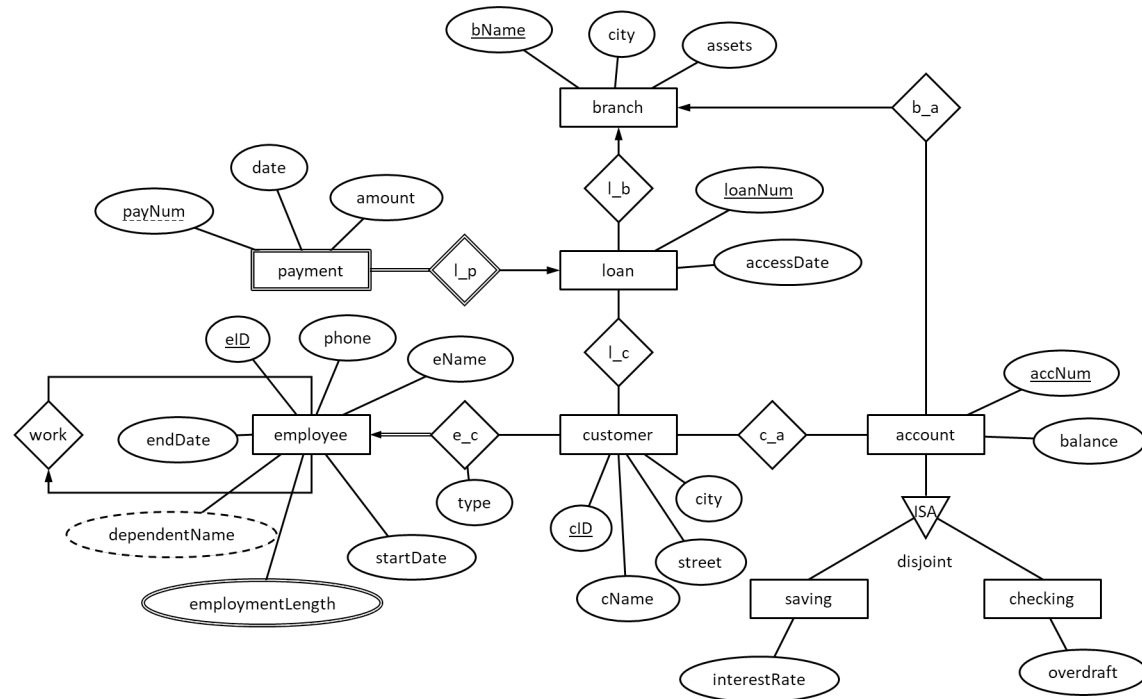
COMP3013 2024 Fall

Assignment 2

Note: there is no requirement on ER diagram drawing. You are allowed to draw ER diagrams using any tool. But you need to make sure that your drawing is clear enough. TAs have rights to remove marks if your graph unreadable. Please do not try to challenge us.

For the submission, please pack all files and convert them into **A SINGLE PDF FILE**. Rename the PDF file as COMP3013_24F_A2_XXXX, where XXXX is your student ID.

Q1. Convert the following ER diagram into logical schemas. (30pt)



- Branch = (bName, city, assets)
- Loan = (loanNum, accessDate, bName)
// bName is a foreign key to Branch.bName.
- Customer = (cID, cName, city, street, type)
- Employee = (eID, eName, phone, startDate, employmentLength, endDate)
- Account = (accNum, balance, type, bName)
// bName is a foreign key to Branch.bName.
- SavingAccount = (accNum, interestRate)
// accNum is a foreign key to Account.accNum.
- CheckingAccount = (accNum, overdraft)
// accNum is a foreign key to Account.accNum.
- Payment = (payNum, date, amount, loanNum)
// loanNum is a foreign key to Loan.loanNum.

- $e_c = (\underline{eID}, \underline{cID})$
 // eID is a foreign key to Employee. eID .
 // cID is a foreign key to Customer. cID .
- $c_a = (\underline{cID}, \underline{accNum})$
 // cID is a foreign key to Customer. cID .
 // $accNum$ is a foreign key to Account. $accNum$.
- $Handles = (\underline{eID}, \underline{loanNum}, endDate)$
 // eID is a foreign key to Employee. eID .
 // $loanNum$ is a foreign key to Loan. $loanNum$.
- $Dependent = (\underline{eID}, dependentName)$
 // eID is a foreign key to Employee. eID .

The schema of a database for public transportation companies is as follows. Keys are underlined.

- $company = (\underline{cID}, cname, address, phone)$
- $route = (\underline{rID}, departure, arrival, \underline{cID})$
 // cID is a foreign key to $company.cID$.
- $vehicle = (\underline{plateNum}, model, capacity, manufacturer, \underline{cID})$
 // cID is a foreign key to $company.cID$.
- $driver = (\underline{dID}, name, gender, age, \underline{cID})$
 // cID is a foreign key to $company.cID$.
- $serve = (\underline{rID}, \underline{plateNum})$
 // rID is a foreign key to $route.rID$.
 // $plateNum$ is a foreign key to $vehicle.plateNum$.
- $drive = (\underline{plateNum}, \underline{dID})$
 // dID is a foreign key to $driver.rID$.
 // $plateNum$ is a foreign key to $vehicle.plateNum$.

Q2. Write a query for each following question. Assuming students' name and instructors' name are unique. (10pt for each)

- a) Find the number of vehicles owned by "Xinhe" (company name).
- b) Find the companies who owns more than 10 vehicles. Show the company names and number of vehicles owned by each.
- c) Find the route which is served by more vehicles than other routes. Show the departure and the arrival of the route.
- d) A driver retires at age 65 for males or at age 60 for females. Remove retired drivers from the database.
- e) The company "ZhuhaiGongjiao" (company name) opens a new route from "UIC" (departure) to "Gongbei" (destination). The company will assign any vehicle manufactured by "BYD" to the new route. Add correct records to the database.

a)

```
SELECT COUNT(*) AS num_vehicles
FROM vehicle v
JOIN company c ON v.cID = c.cID
WHERE c.cname = 'Xinhe';
```

b)

```
SELECT c.cname, COUNT(*) AS num_vehicles
FROM vehicle v
```

```

JOIN company c ON v.cID = c.cID
GROUP BY c.cID, c.cname
HAVING COUNT(*) > 10;1
c)
SELECT r.departure, r.arrival
FROM route r
JOIN serve s ON r.rID = s.rID
GROUP BY r.rID, r.departure, r.arrival
ORDER BY COUNT(s.plateNum) DESC
LIMIT 1;
d)
DELETE FROM
WHERE (gender = 'male' AND age >= 65) or (gender = 'female'
AND age >= 60);
e)
INSERT INTO route (rID, departure, arrival, cID)
VALUES (
    (SELECT MAX(rID) + 1 FROM route),
    'UIC',
    'Gongbei',
    (SELECT cID FROM company WHERE cname = 'ZhuhaiGongjiao')
);

INSERT INTO serve (rID, plateNum)
SELECT
    (SELECT rID FROM route WHERE departure = 'UIC' AND
    arrival = 'Gongbei' AND cID = (SELECT cID FROM company
    WHERE cname = 'ZhuhaiGongjiao')),
    plateNum
FROM vehicle
WHERE manufacturer = 'BYD';

```

Q3. Implement the above logical design in SQL. You need to select suitable data types and link foreign keys properly. (10 pt)

```

-- Table: company
CREATE TABLE company (
    cID INT PRIMARY KEY,
    cname VARCHAR(100) NOT NULL,
    address VARCHAR(255),
    phone VARCHAR(15)
);

-- Table: route
CREATE TABLE route (
    rID INT PRIMARY KEY,
    departure VARCHAR(100) NOT NULL,
    arrival VARCHAR(100) NOT NULL,
    cID INT NOT NULL,
    FOREIGN KEY (cID) REFERENCES company(cID)
);

```

```

-- Table: vehicle
CREATE TABLE vehicle (
    plateNum VARCHAR(20) PRIMARY KEY,
    model VARCHAR(50) NOT NULL,
    capacity INT,
    manufacturer VARCHAR(100),
    cID INT NOT NULL,
    FOREIGN KEY (cID) REFERENCES company(cID)
);

-- Table: driver
CREATE TABLE driver (
    dID INT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    gender ENUM('male', 'female') NOT NULL,
    age INT,
    cID INT NOT NULL,
    FOREIGN KEY (cID) REFERENCES company(cID)
);

-- Table: serve
CREATE TABLE serve (
    rID INT NOT NULL,
    plateNum VARCHAR(20) NOT NULL,
    PRIMARY KEY (rID, plateNum),
    FOREIGN KEY (rID) REFERENCES route(rID),
    FOREIGN KEY (plateNum) REFERENCES vehicle(plateNum)
);

-- Table: drive
CREATE TABLE drive (
    plateNum VARCHAR(20) NOT NULL,
    dID INT NOT NULL,
    PRIMARY KEY (plateNum, dID),
    FOREIGN KEY (plateNum) REFERENCES vehicle(plateNum),
    FOREIGN KEY (dID) REFERENCES driver(dID)
);

```

Q4. The above schema is converted from an ER diagram which has *company*, *route*, *vehicle*, and *driver* as entity sets; *serve* and *drive* as relationship sets. What are the constraints on the two relationship sets? (10 pt)

1. Serve Relationship (route ↔ vehicle)

Cardinality: Many-to-Many

Each route can be served by multiple vehicles, and each vehicle can serve multiple routes.

Participation: Partial participation

Not all routes may have vehicles assigned, and not all vehicles may serve routes at all times.

Primary Key:

Combination of `rID` and `plateNum` ensures that a specific vehicle serves a specific route uniquely.

Foreign Keys:

`rID` references `route.rID`.

`plateNum` references `vehicle.plateNum`.

2. Drive Relationship (vehicle ↔ driver)

Cardinality: Many-to-Many

A vehicle can have multiple drivers over time, and a driver can operate multiple vehicles.

Participation: Partial participation

Not all drivers may be assigned to vehicles at all times, and not all vehicles may have drivers assigned.

Primary Key:

Combination of `plateNum` and `dID` ensures that a specific driver operates a specific vehicle uniquely.

Foreign Keys:

`plateNum` references `vehicle.plateNum`.

`dID` references `driver.dID`.